Kỹ Thuật Đệ Quy Quay Lui (Backtracking)

Tấn Đức

August 12, 2026

Nội dung bài học

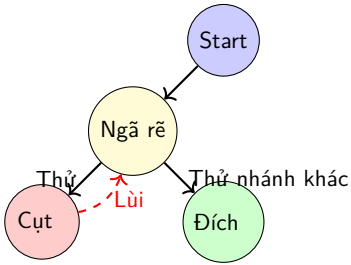
1. Dẫn nhập & Bản chất Quay lui
2. Bài toán 1: Sinh xâu nhị phân
3. Bài toán 2: Dãy con có tổng bằng S (Có Cắt Tỉa)
4. Bài toán 3: Xếp hậu (N-Queens)
5. Tổng kết & Phân tích

Quay lui (Backtracking) là gì?

Khái niệm trực quan (Dành cho người mới)

Hãy tưởng tượng bạn đang tìm đường ra khỏi một mê cung:

- Đi dọc theo một con đường đến khi gặp **ngã rẽ**.
- Chọn một hướng để đi tiếp.
- Nếu đụng **ngõ cụt** (sai), bạn quay lại ngã rẽ vừa rồi.
- Chọn hướng khác và tiếp tục thử.



Lưu ý

Quay lui chính là **thử và sai**. Ta xây dựng lời giải từng bước, nếu thấy bước hiện tại không thể dẫn tới đích, ta HỦY BỎ bước đó và thử lựa chọn khác.

Mô hình mã giả (Pseudocode) tổng quát

Ý tưởng - Cấu trúc cốt lõi

Mọi bài toán Backtracking đều xoay quanh một hàm đệ quy sinh ra các phần tử của cấu hình nghiệm. Giả sử nghiệm cần tìm là một mảng $X = [x_1, x_2, \ldots, x_n]$.

Mô hình mã giả (Pseudocode) tổng quát

```
void Try(int i) {
    for (moi gia tri V co the gan cho X[i]) {
        if (V hop le) {
            X[i] = V; // Thu chon V cho vi tri i

            if (i == n) {
                // Da xay dung du n phan tu -> Phat hien 1 nghiem
                InKetQua();
            } else {
                // Chua du -> Di tiep xay dung vi tri i+1
                Try(i + 1);
            }

            // QUAN TRONG: Xoa bo lua chon, "Quay lui" (Backtrack)
            // Trong nhieu bai toan, buoc nay giup phuc hoi trang thai
        }
    }
}
```

Bài toán: Sinh xâu nhị phân độ dài N

Phát biểu bài toán

Cho số nguyên dương N. Hãy in ra tất cả các xâu nhị phân (chỉ gồm 0 và 1) có độ dài đúng bằng N, theo thứ tự từ điển.

Input: $N = 3$

Output: 000, 001, 010, 011, 100, 101, 110, 111

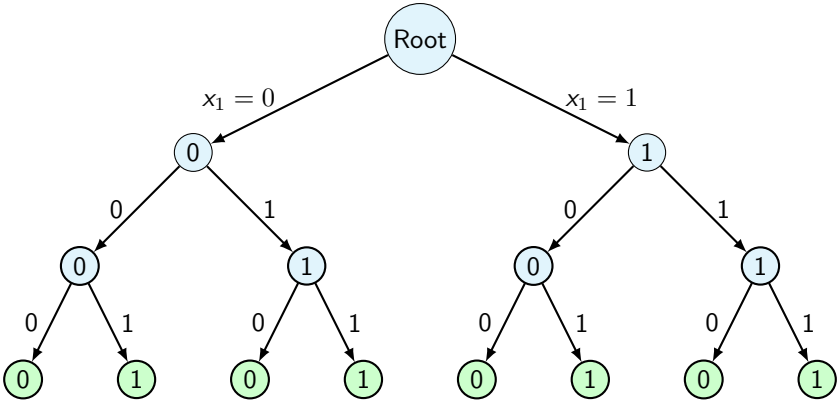
Ý tưởng

- Xâu có dạng: $X = x_1 x_2 \dots x_N$
- Mỗi vị trí x_i có đúng 2 lựa chọn: 0 hoặc 1.
- Ta sẽ dùng hàm `Try(i)` để điền giá trị cho vị trí i.

Mẹo: Về thứ tự sinh

Để kết quả in ra đúng thứ tự từ điển, trong vòng lặp ta luôn thử số 0 trước, sau đó mới thử số 1.

Minh họa Không gian trạng thái (Cây tìm kiếm) với N=3



Khi duyệt đến node lá (màu xanh lá), ta thu được 1 cấu hình hoàn chỉnh.

Mã nguồn C++: Sinh xâu nhị phân

```cpp
int N;
int X[100]; // Mang luu cau hinh
void Print() {
    for (int i = 1; i <= N; ++i) cout << X[i];
    cout << "\n";
}

void Try(int i) {
    // Ung cu vien cho vi tri i la 0 hoac 1
    for (int v = 0; v <= 1; ++v) {
        X[i] = v; // Ghi nhan lua chon
        if (i == N) {
            Print();  // Xay dung du N phan tu, in ra
        } else {
            Try(i + 1); // De quy di tiep xau dung vi tri i+1
        }
    }
}
```

Bài toán: Dãy con có tổng bằng S

Phát biểu bài toán

Cho mảng A gồm N số nguyên dương và số S. Hãy in ra tất cả các tập con của A sao cho tổng các phần tử trong tập con đúng bằng S.

Input: $A = \{2, 3, 5, 1\}$, $S = 5$

Output: $\{2, 3\}, \{5\}$

Ý tưởng

Bài toán này thực chất là biến thể của sinh xâu nhị phân!

- $x_i = 1$ nghĩa là **CHỌN** phần tử $A[i]$ vào tập con.
- $x_i = 0$ nghĩa là **BỎ QUA** phần tử $A[i]$.
- Ở mỗi bước, ta duy trì một biến `sum` lưu tổng các phần tử đã chọn.

Kỹ thuật nhánh cận

Vấn đề của thuật toán ngây thơ (Naïve)

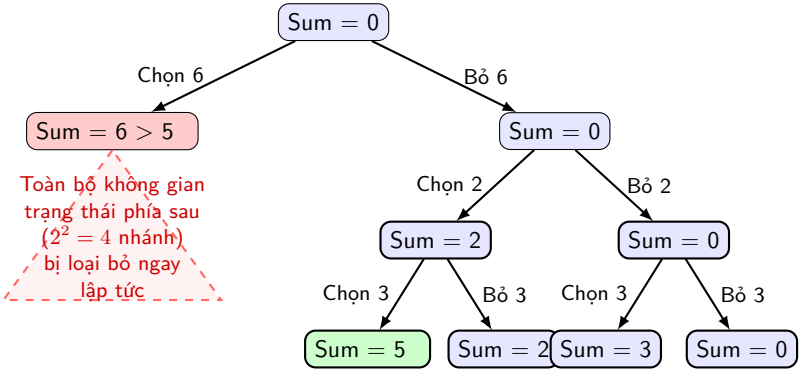
Nếu sinh hết 2^N xâu nhị phân rồi mới kiểm tra tổng có bằng S không thì cực kỳ chậm.

Áp dụng nhánh cận

Vì mảng A chỉ chứa **số nguyên dương**, tổng các phần tử khi thêm vào sẽ chỉ TĂNG LÊN. Do đó, nếu tại một bước nào đó, `sum` > `S` thì ta lập tức **DỪNG (Cắt nhánh)** mà không cần duyệt sâu thêm nữa vì chắc chắn vô nghiệm!

Nhánh cận = Ngừng thử khi biết chắc chắn sai.

Minh họa Sức mạnh Nhánh cận ($A = \{6, 2, 3\}$, $S = 5$)



Với $A = \{6, 2, 3\}$, ta thấy rõ: Tại ngay **mức 1**, khi chọn phần tử 6, tổng đã vượt quá S. Thuật toán lập tức quay lui, **không cần sinh ra** các trường hợp chọn/bỏ 2 và 3 phía dưới, giúp giảm đáng kể thời gian chạy!

Mã nguồn C++: Tập con có tổng bằng S

```cpp
void Try(int i, int cur_sum) {
    if (cur_sum > S) return; // CAT TIA (Pruning)
    if (i > N) {
        if (cur_sum == S) {
            for (int j = 1; j <= N; ++j)
                if (X[j] == 1) cout << A[j] << " ";
            cout << "\n";
        }
        return;
    }

    // Nhanh 0: Khong chon A[i]
    X[i] = 0; Try(i + 1, cur_sum);
    // Nhanh 1: Chon A[i]
    X[i] = 1; Try(i + 1, cur_sum + A[i]);
}
```

Bài toán: Xếp N quân hậu

Phát biểu bài toán kinh điển

Cho bàn cờ vua kích thước $N \times N$. Hãy tìm cách đặt N quân hậu lên bàn cờ sao cho **không có bất kỳ hai quân hậu nào có thể ăn nhau**.
(Hậu ăn theo hàng ngang, cột dọc và 2 đường chéo).

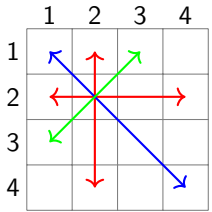
Ý tưởng

- Không thể đặt 2 quân hậu trên cùng 1 hàng. Do đó, chắc chắn mỗi hàng sẽ có đúng 1 quân hậu.
- Hàm `Try(i)`: Tìm vị trí cột `j` để đặt quân hậu trên hàng `i`.
- Trước khi đặt, phải kiểm tra: Cột `j`, đường chéo chính và đường chéo phụ đã bị chiếm chưa?

Phân tích Bàn cờ và Xung đột

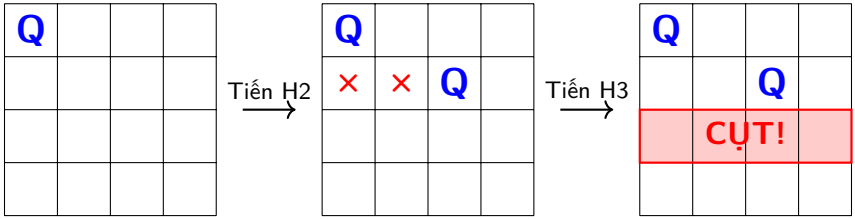
Để truy vấn $O(1)$ xem vị trí (i, j) có an toàn không, ta dùng 3 mảng đánh dấu:

- `col[j]`: Đánh dấu cột j.
- `d1[i - j + N]`: Đường chéo chính ($\searrow$). Chênh lệch $i - j$ là hằng số. Ta cộng thêm N để tránh chỉ số âm.
- `d2[i + j - 1]`: Đường chéo phụ ($\nearrow$). Tổng $i + j$ là hằng số.



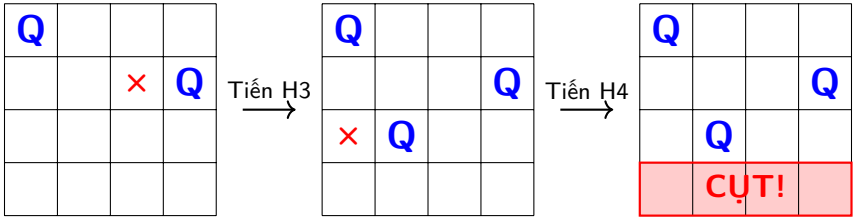
Hậu tại Hàng 2, Cột 2 khống chế: Cột 2, Hàng 2, Chéo chính, Chéo phụ.

Minh họa Trực quan Quay lui (N=4) - Cụt ở Hàng 3



Ý tưởng

Minh họa Trực quan Quay lui (N=4) - Cụt ở Hàng 4



B4: Ở Hàng 2, dời Q sang H2C4

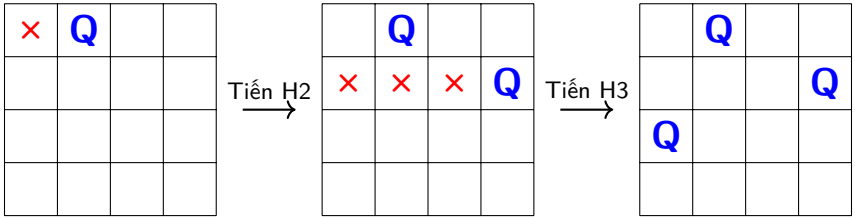
B5: H3C1 bị khống chế. Đặt Q ở H3C2

B6: Toàn bộ Hàng 4 bị khống chế.

Backtrack! Hủy H3C2

Mẹo: Backtrack liên tiếp

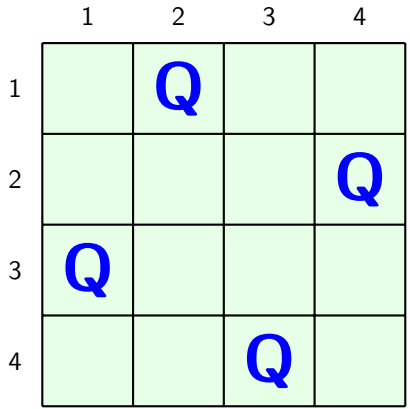
Minh họa Trực quan Quay lui (N=4) - Nhánh đúng



Ý tưởng

Nhờ việc dời quân hậu đầu tiên sang cột 2, không gian các cột còn lại mở rộng hơn, tạo điều kiện thuận lợi cho các hàng tiếp theo.

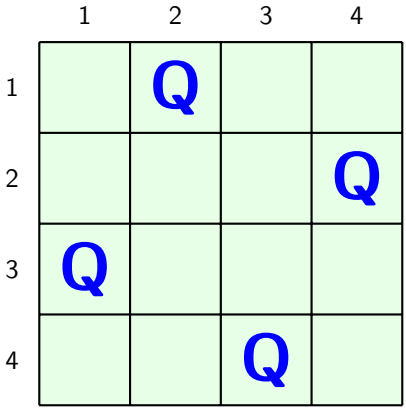
Minh họa Trực quan Quay lui (N=4) - Tìm thấy nghiệm!



Tổng kết

- **Bước cuối cùng:** Tại Hàng 4, ô ở Cột 3 hoàn toàn an toàn (Không trùng cột, không trùng 2 đường chéo với bất kỳ Hậu nào phía trên).
- **Nghiệm đầu tiên:** `X = [2, 4, 1, 3]` (H1C2, H2C4, H3C1, H4C3).
- Quá trình đệ quy đạt đến `i = N = 4`. Kết quả được in ra.

Minh họa Trực quan Quay lui (N=4) - Tìm thấy nghiệm!



Mẹo: Hành trình chưa dừng lại

Sau khi in ra nghiệm, thuật toán sẽ **tiếp tục Backtrack** (hủy ô H4C3, lùi về H3...) để quét toàn bộ không gian nhằm tìm ra tất cả các nghiệm khác của bài toán!

Mã nguồn C++: Thuật toán N-Queens

```cpp
int N = 8, ans = 0;
int X[20]; // X[i] = j: Hang i dat hau o cot j
bool col[20], d1[40], d2[40]; // Mang danh dau
void Try(int i) {
    for (int j = 1; j <= N; ++j) {
        // Neu cot, duong cheo chinh, cheo phu deu an toan
        if (!col[j] && !d1[i - j + N] && !d2[i + j - 1]) {
            // Danh dau la da chiem
            X[i] = j;
            col[j] = d1[i - j + N] = d2[i + j - 1] = true;

            if (i == N) ans++; // Tim thay 1 cach xep
            else Try(i + 1);   // De quy xep hang tiep theo

            // BACKTRACK: Bo danh dau (khoi phuc trang thai)
            col[j] = d1[i - j + N] = d2[i + j - 1] = false;
        }
    }
}
```

Tổng kết về kỹ thuật Quay lui

Tổng kết

- Backtracking là quá trình xây dựng lời giải **từng phần một**.
- Luôn phải có bước **khôi phục trạng thái** (Quay lui) trừ khi bài toán không chia sẻ trạng thái chung.
- Có thể áp dụng cho mọi bài toán Liệt kê, Hoán vị, Tổ hợp, Tìm đường.

Độ phức tạp

- **Độ phức tạp Thời gian:** Thường là hàm mũ $O(2^N)$, giai thừa $O(N!)$. Chỉ chạy được với N nhỏ ($N \leq 20$).
- **Độ phức tạp Không gian:** $O(N)$ độ sâu của Stack đệ quy.